

Using Machine Learning Methodologies to Predict Heart Disease

Abstract

Heart disease is a broad classification covering many types of heart-affecting conditions. Heart disease is the single leading cause of death in Australia. This report will examine the link between 11 categorical and numeric variables and the presence of heart disease in a patient. The dataset is available online from the Institute of Electrical and Electronics Engineers and is a compilation of common variables across five other data sets. The dataset will be examined using the supervised methods of the Naïve Bayes Classifier, logistics regression, KNN classification models, and an unsupervised k-means clustering model. This report found acceptable levels of accuracy, greater than 0.7, from all implemented supervised learning methods, utilising three different sets of features for the logistics regression and KNN classification models: all features, significant features from the “all features” model and Principal Component Analysis. The k-means clustering did not provide an acceptable explanation of the dataset variance, 36.5%. Thus, it appears possible to accurately create a model that predicts the presence of heart disease using this data set’s features. Further study would help quantify the significant features and validate the model on more observations.

Introduction

In 2017, 18,590 deaths were attributed to heart disease in Australia. Heart disease is a broad classification for many conditions that affect the heart, including but not limited to the blood vessels, congenital heart defects, arrhythmias, and coronary artery disease (“*What is heart disease*”, 2021). The prognosis of many forms of heart disease is improved through early detection. Thus, it is essential to detect and, if possible, predict the presence of heart disease in patients (“*Early heart disease detection saves lives*”, 2021).

Machine Learning presents an opportunity to this field, with the ability to utilise algorithms and previously unconsidered features, to quickly assess multiple models and methodologies to discover the most accurate models to detect patients with heart disease.

This report aims to examine some common predictors of heart disease and develop a model that uses these features to predict the presence of heart disease. This report will review the data set, features, and response variables. Then these variables will be used in the creation of varied models, which will utilise both supervised and unsupervised learning techniques.

Data

This work uses an open-access dataset accessible from the Institute of Electrical and Electronics Engineers (IEEE). This dataset is the result of curation and a combination of the common features of five other datasets publicly available datasets.

The initial data set consists of 1190 observations of 12 variables. 11 of these variables are features, with the final variable, “target”, as the response variable. A binary classification where a value of 0 indicates a patient does not have heart disease, and a value of 1 means the patient does (Siddhartha, 2020).

Table 1. Variable Summary

Attribute	Variable Name	Data Type	Description
Age	age	Numeric	Age in years
Sex	sex	Binary	0 = female 1 = male
Chest Pain Type	chest pain type	Nominal	1 = typical angina 2 = atypical angina 3 = non-angina pain 4 = asymptomatic
Resting Blood Pressure	resting bp s	Numeric	In mmHg
Serum Cholesterol	cholesterol	Numeric	In mg/dL
Fasting Blood Sugar	fasting blood sugar	Binary	0 = <120mg/dL 1 = ≥120mg/dL
Resting Electrocardiogram Results	resting ecg	Nominal	0 = Normal 1 = ST-T wave abnormality 2 = left ventricular hypertrophy by Estes' criteria
Maximum Heart Rate Achieved	max heart rate	Numeric	Number in range 60 - 202
Exercise Induced Angina	exercise induced angina	Binary	0 = no 1 = yes
Oldpeak = ST	oldpeak	Numeric	Number in range -2.6 - 6.2
The slope of the peak Exercise ST Segment	st slope	Nominal	0, 1 = upslope 2 = flat 3 = downslope
Class (response variable)	target	Binary	0 = normal 1 = heart disease

Before use in this work, the main known preprocessing of the dataset is the combination of other publicly available datasets into a single dataset by IEEE. Examining the variables, there was likely other preprocessing to make the variable consistent across datasets. The sex variable was likely different across datasets (male/female vs 0,1), and there was pre-processing to convert the fasting blood sugar variable from a numeric variable into a binary variable.

Methods

This dataset's features and response variable lend it to multiple types of supervised and unsupervised learning due to the mix of categorical and numeric variables and the binary categorical response variable. The first model will use the Naïve Bayes Classifier; this model will provide a baseline level of accuracy for further models, as the Naïve Bayes Classifier is useful at predicting a response utilising a variety of feature types.

For supervised learning, logistic regression and KNN classification models will be examined to find the most accurate model.

Two dimensionality reduction methods will be applied to the data, and model accuracy will be assessed again. The first method of dimensionality reduction will be Feature Selection. Only the most significant features of the original model will be used. These features will be those with a p-value ≥ 0.05 . The second dimensionality reduction method will be Principal Component Analysis, an unsupervised method of Feature Extraction. The PCA will then be assessed for the correct number of Principal Components to include in a third version of the GLM/KNN models. Then, all seven models (Naïve Bayes Classifier and the three versions of GLM and KNN) will be assessed to determine which model proves to be the most accurate and robust.

Finally, k-means clustering will be used to assess the dataset for the natural clusters in the dataset. Ideally, these clusters in the data will separate by the response variable.

All analysis of this dataset was performed using R Studio version 2022.7.2.576. All R Codes and outputs are available in Appendix 1.

Results and Discussion

Naïve Bayes Classifier

Linear Discriminant Analysis and Quadratic Discriminant Analysis were initially considered using the numeric features subset. However, these models were deemed unacceptable due to violations of the assumptions of normality and homoscedasticity. Thus, the Naïve Bayes Classifier (NBC) was used. This model provides an additional benefit in also being able to utilise the categorical variables within the dataset. The dataset was then examined for the assumptions required by the NBC.

1) Independence of Features

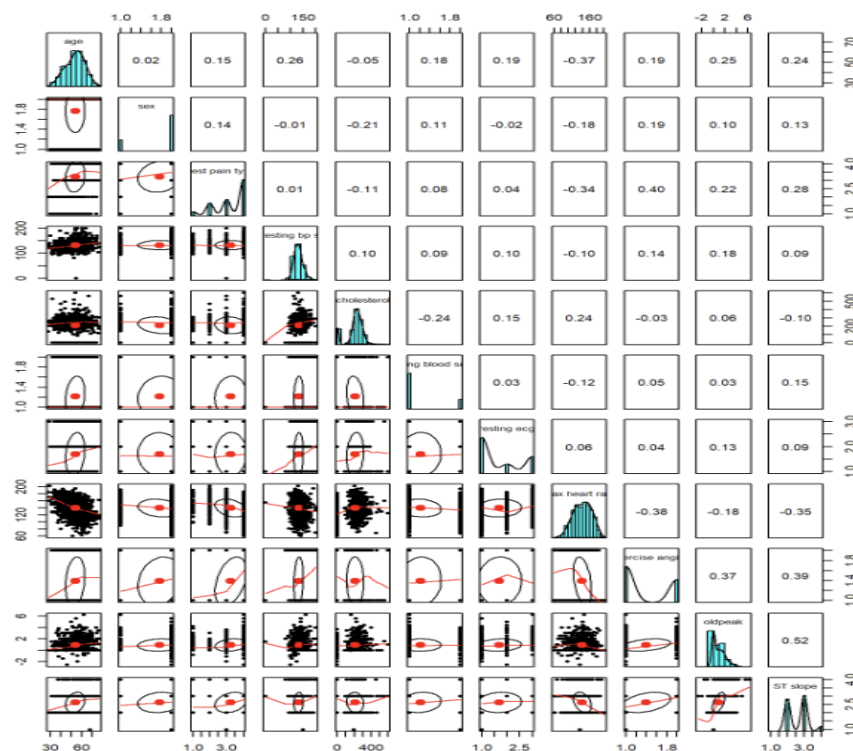


Figure 1 Pairwise Plots of Variables

The pair plots in Figure 1 show the max correlation coefficient is 0.52 between the “oldpeak” and “ST slope” variables. Anything above 0.5 or below -0.5 could be considered a relationship. Although a value this low is not conclusive evidence of non-independence of these variables, NBC typically exhibits good performance against violations of this assumption.

2) Equality of Features

To perform a Naive Bayes Classifier, the assumption is that all variables have an equal effect on the model’s outcome. It can be assumed that these features will have an equal effect on the outcome.

Thus, with both assumptions met, an NBC for this dataset will be trained using a random subset of 80% of the dataset and then applied to the test data.

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 85 26
##           1 27 99
##
##           Accuracy : 0.7764
##           95% CI : (0.7179, 0.8278)
##           No Information Rate : 0.5274
##           P-Value [Acc > NIR] : 1.928e-15
##
##           Kappa : 0.5512
##
##   Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.7589
##           Specificity : 0.7920
##           Pos Pred Value : 0.7658
##           Neg Pred Value : 0.7857
##           Prevalence : 0.4726
##           Detection Rate : 0.3586
##           Detection Prevalence : 0.4684
##           Balanced Accuracy : 0.7755
##
##           'Positive' Class : 0
##
```

Figure 2 Confusion Matrix of NBC model on test data

Viewing the confusion matrix for the application of the NBC against the test data, it is evident that this model provides good performance, with an accuracy of 0.7764, with industry standards being between 0.7 and 0.9. This value indicates the model does not suffer from over-fitting and thus will apply to other data sets which utilise the same features. Examining the prediction results and the sensitivity (true normal patient) and the specificity (true heart disease patient), the model is slightly better at predicting the presence of heart disease (0.7920) than ‘normal’ (0.7589).

It is important to note that these values will change depending on the specific partitioning of the training and test sets.

GLM and KNN regression/classification

Examining the results of the GLM model in Figure 3, it is evident that not all features are as important in determining the outcome. Only the features with a p-value of <0.05 will be used to reduce dimensionality in the model and maintain model accuracy. These features are “sex”, “chest pain type”, “cholesterol”, “fasting blood sugar”, “exercise angina”, and “oldpeak”. The high p-values in the ST slope variable indicate multicollinearity and, thus, is also likely to be a significant feature.

```
##
## Call:
## NULL
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.7738  -0.4219   0.1785   0.4634   2.7319
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    12.153461  535.411540   0.023 0.981890
## sex1           1.495438   0.254810   5.869 4.39e-09 ***
## '\'\chest pain type\\'2\' -0.318861   0.456491  -0.699 0.484861
## '\'\chest pain type\\'3\' -0.143196   0.407533  -0.351 0.725309
## '\'\chest pain type\\'4\'  1.625650   0.395542   4.110 3.96e-05 ***
## cholesterol    -0.004407   0.001085  -4.061 4.89e-05 ***
## '\'\fasting blood sugar\\'1\'  0.919708   0.261561   3.516 0.000438 ***
## '\'\exercise angina\\'1\'    0.846917   0.225068   3.763 0.000168 ***
## oldpeak        0.483927   0.107921   4.484 7.32e-06 ***
## '\'\ST slope\\'1\'      -15.055969  535.411260  -0.028 0.977566
## '\'\ST slope\\'2\'      -12.831152  535.411265  -0.024 0.980880
## '\'\ST slope\\'3\'      -14.119177  535.411391  -0.026 0.978962
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1318.0  on 952  degrees of freedom
## Residual deviance:  663.4  on 941  degrees of freedom
## AIC: 687.4
##
## Number of Fisher Scoring iterations: 12
```

Figure 3 GLM model

New models were then generated that only utilised these features.

Principal Component Analysis

An essential factor in producing a machine-learning model is dimensionality reduction. The four main problems with high dimensionality are high computational cost, over-fitting of the model on training data, multicollinearity, and high dimensions can lead to inaccuracy in some distance-based models. Previously, one form of dimensionality reduction was used to select significant features. Another form of dimensionality reduction is Principal Component Analysis, an unsupervised form of Feature Extraction (Abhigyan, 2020).

Three methods of determining the number of PCs were used—examination for an elbow in the scree plot, the Kaiser rule, and Horn’s Parallel Analysis. Examination of the scree plot for an elbow was inconclusive (Figure 4).

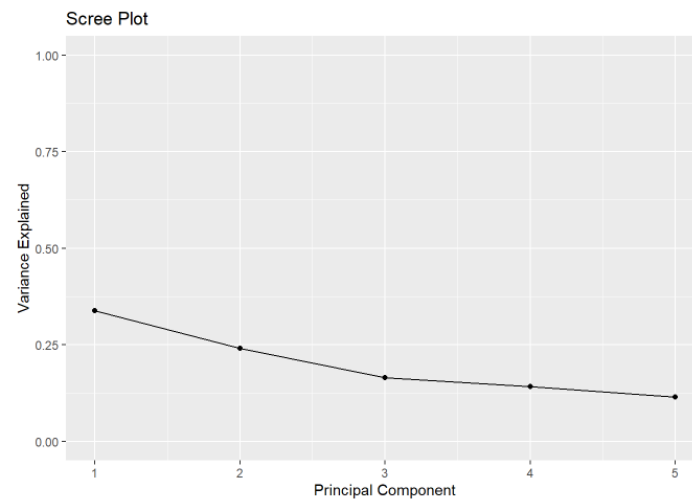


Figure 4 Scree Plot of variance explained by Principal Components.

The Kaiser rule is to use PCs with eigenvalues greater than 1, which indicates PCs that provide more meaningful information than a single variable (“Kaiser rule”, n.d.), and Horns Parallel analysis both indicated the use of two Principal Components. Figure 5 shows a Scree Plot of the Eigenvalues of each principal component and Horn’s Parallel Analysis (“Using Horn’s parallel analysis method in exploratory factor analysis for determining the number of factors”, 2016).

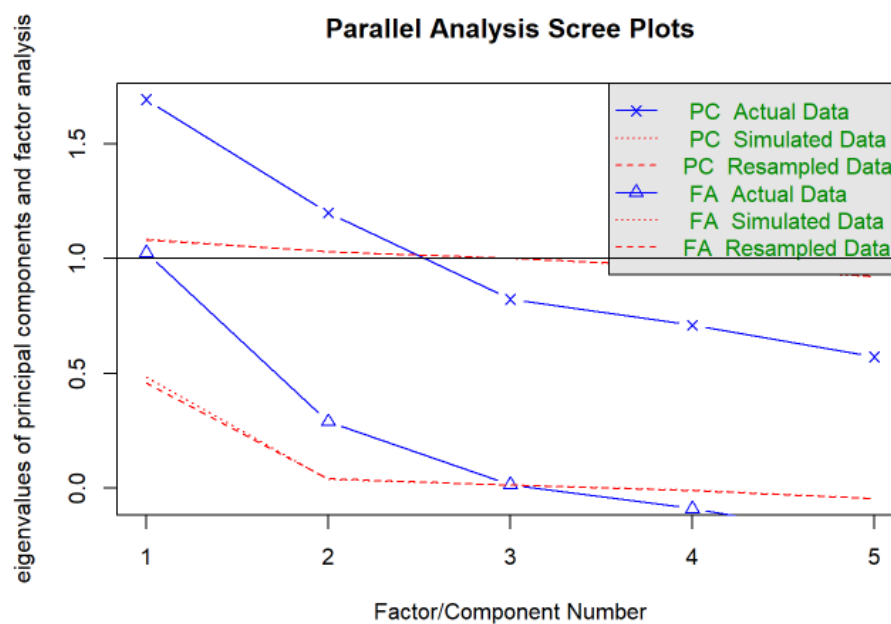


Figure 5 Parallel Analysis Scree Plots (Eigenvalue of Principal Components)

Thus, this dataset's first two Principal Components were used to train a new model.

Summary of Model Accuracy on Test Data

Before examining the accuracy of the models, it is essential to reiterate that these values are subject to change with different partitions of the data into training and testing sets. A table summary of the accuracy and kappa values of the seven models on the test data sets is below.

Table 2. Accuracy of Models

Features Used	Methodology	Model Accuracy on Test Data	Model Kappa value on Test Data
All	NBC	0.7763713	0.5511845
	GLM	0.8438819	0.6860927
	KNN	0.7046414	0.4063976
Significant Features	GLM	0.8354430	0.6691248
	KNN	0.7299578	0.4562661
Principal Components 1 and 2	GLM	0.7046414	0.4047363
	KNN	0.7215190	0.4392744

glm.conf	glm.sig.conf	glm.pca.conf
<pre>## Confusion Matrix and Statistics ## ## Reference ## Prediction 0 1 ## 0 91 16 ## 1 21 109 ## ## Accuracy : 0.8439 ## 95% CI : (0.7913, 0.8876) ## No Information Rate : 0.5274 ## P-Value [Acc > NIR] : <2e-16 ## ## Kappa : 0.6861 ## ## Monemar's Test P-Value : 0.5108 ## ## Sensitivity : 0.8125 ## Specificity : 0.8720 ## Pos Pred Value : 0.8505 ## Neg Pred Value : 0.8385 ## Prevalence : 0.4726 ## Detection Rate : 0.3840 ## Detection Prevalence : 0.4515 ## Balanced Accuracy : 0.8422 ## ## 'Positive' Class : 0 ##</pre>	<pre>## Confusion Matrix and Statistics ## ## Reference ## Prediction 0 1 ## 0 90 17 ## 1 22 108 ## ## Accuracy : 0.8354 ## 95% CI : (0.782, 0.8803) ## No Information Rate : 0.5274 ## P-Value [Acc > NIR] : <2e-16 ## ## Kappa : 0.6691 ## ## Monemar's Test P-Value : 0.5218 ## ## Sensitivity : 0.8036 ## Specificity : 0.8640 ## Pos Pred Value : 0.8411 ## Neg Pred Value : 0.8308 ## Prevalence : 0.4726 ## Detection Rate : 0.3797 ## Detection Prevalence : 0.4515 ## Balanced Accuracy : 0.8338 ## ## 'Positive' Class : 0 ##</pre>	<pre>## Confusion Matrix and Statistics ## ## Reference ## Prediction 0 1 ## 0 72 30 ## 1 40 95 ## ## Accuracy : 0.7046 ## 95% CI : (0.6421, 0.7619) ## No Information Rate : 0.5274 ## P-Value [Acc > NIR] : 1.986e-08 ## ## Kappa : 0.4047 ## ## Monemar's Test P-Value : 0.2821 ## ## Sensitivity : 0.6429 ## Specificity : 0.7600 ## Pos Pred Value : 0.7059 ## Neg Pred Value : 0.7037 ## Prevalence : 0.4726 ## Detection Rate : 0.3038 ## Detection Prevalence : 0.4304 ## Balanced Accuracy : 0.7014 ## ## 'Positive' Class : 0 ##</pre>

Figure 7 Confusion Matrices of GLM models (left to right: all features, significant features, first two Principal Components)

knn.conf	knn.sig.conf	knn.pca.conf
<pre>## Confusion Matrix and Statistics ## ## Reference ## Prediction 0 1 ## 0 75 33 ## 1 37 92 ## ## Accuracy : 0.7046 ## 95% CI : (0.6421, 0.7619) ## No Information Rate : 0.5274 ## P-Value [Acc > NIR] : 1.986e-08 ## ## Kappa : 0.4064 ## ## Monemar's Test P-Value : 0.7199 ## ## Sensitivity : 0.6696 ## Specificity : 0.7360 ## Pos Pred Value : 0.6944 ## Neg Pred Value : 0.7132 ## Prevalence : 0.4726 ## Detection Rate : 0.3165 ## Detection Prevalence : 0.4557 ## Balanced Accuracy : 0.7028 ## ## 'Positive' Class : 0 ##</pre>	<pre>## Confusion Matrix and Statistics ## ## Reference ## Prediction 0 1 ## 0 76 28 ## 1 36 97 ## ## Accuracy : 0.73 ## 95% CI : (0.6687, 0.7854) ## No Information Rate : 0.5274 ## P-Value [Acc > NIR] : 1.345e-10 ## ## Kappa : 0.4563 ## ## Monemar's Test P-Value : 0.3816 ## ## Sensitivity : 0.6786 ## Specificity : 0.7760 ## Pos Pred Value : 0.7308 ## Neg Pred Value : 0.7293 ## Prevalence : 0.4726 ## Detection Rate : 0.3207 ## Detection Prevalence : 0.4388 ## Balanced Accuracy : 0.7273 ## ## 'Positive' Class : 0 ##</pre>	<pre>## Confusion Matrix and Statistics ## ## Reference ## Prediction 0 1 ## 0 75 29 ## 1 37 96 ## ## Accuracy : 0.7215 ## 95% CI : (0.6598, 0.7776) ## No Information Rate : 0.5274 ## P-Value [Acc > NIR] : 7.713e-10 ## ## Kappa : 0.4393 ## ## Monemar's Test P-Value : 0.3889 ## ## Sensitivity : 0.6696 ## Specificity : 0.7680 ## Pos Pred Value : 0.7212 ## Neg Pred Value : 0.7218 ## Prevalence : 0.4726 ## Detection Rate : 0.3165 ## Detection Prevalence : 0.4388 ## Balanced Accuracy : 0.7188 ## ## 'Positive' Class : 0 ##</pre>

Figure 6 Confusion Matrices of KNN models (left to right: all features, significant features, first two Principal Components)

The industry standard for machine learning models is an accuracy of 0.7 to 0.9 (70 – 90%) (FutureAnalytics, 2022). All these models have an accuracy value greater than 0.7, indicating that these are all acceptable models. However, the GLM models using the primary features are the only models with accuracy values above 80%. The GLM model utilising all features has the best accuracy (0.8438819), with the GLM model utilising the significant features next (0.8354430). The other five models provide accuracy in the 70-80% range.

Interestingly, while the GLM models' accuracy decreases with dimensionality reduction, the KNN models' accuracy increases with dimensionality reduction. This is likely due to the Euclidean nature of the KNN algorithm, with required numeric data. This results in a model that cannot utilise almost half of the features in the dataset. Therefore, the accuracy can increase with fewer categorical variables in the significant features model, and the PCA models have only numeric data. Therefore, the reverse is true of GLM, which can utilise those categorical variables.

The accuracy statistic is not the only statistic of interest. The Kappa statistic measures the targets in the data vs. the values the model ascribed. The same trend across models is seen as the accuracy; the GLM model with all features has the best kappa value (0.6860927), the GLM model using significant features is next with a kappa value of (0.6691248), with the rest having kappa values of less than 0.6.

Thus, while all the models provide acceptable accuracy, the difference in the value of the kappa statistic is more significant. Considering a kappa value of 0.81 to 1 to indicate almost perfect agreement, 0.61 to 0.8 indicating substantial agreement and 0.41 to 0.60 as moderate agreement (Landis & Koch, 1977), the best models are the GLM model with all features and the GLM model with just the significant features, as they have similarly high accuracy and kappa values indicating substantial agreement between the model and the known values of the target in the dataset.

K-means Clustering

With the use of the Naive Bayes Classifier and the classification/regression models (GLM/KNN, GLM/KNN with significant features, GLM/KNN with Principal Components), the goal was to examine the data to discover a classification methodology of whether a patient has heart disease or not, and this somewhat succeeded, the best model, GLM with significant features returned a model that classified patients with approximately 82% accuracy. Interestingly, while the data contains a binary classification (normal or heart disease), the KNN models found other clusters. In the original model and PC models, $k = 9$, and in the significant figures model, $k=5$.

It is essential for the best patient outcomes to discover whether these different clusters indicate differences in indicators where the patient does not have heart disease vs differences in the indicators where the patient does have heart disease or whether these clusters could be indicative of a state where heart disease is not yet present but is likely.

Thus, cluster analysis will be performed. Before choosing a clustering methodology, the most crucial consideration is that the size and shapes of the clusters are unknown. Therefore k-means clustering will be utilised.

The data must be scaled initially. This is because k-means clustering is a distance-based algorithm, thus biased towards variables with higher magnitudes. Scaling the data prevents this by ensuring all variables have the same magnitude.

At this point, the optimal number of clusters (k) must be determined. This is performed by examining the elbow in a Sum of Squared errors (SSE) plot. If this is inconclusive, the Silhouette Width Criterion can be used to determine k using the greatest value.

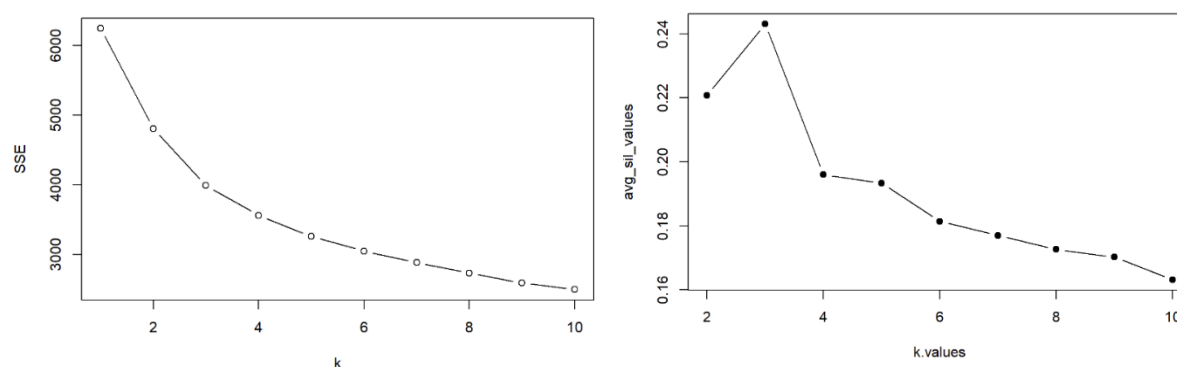


Figure 8 Optimal k -value analysis (left = Sum of Squared Errors, right = Silhouette Width Criterion)

An examination of Figure 8 for the elbow of the SSE graph appears to be at either 3 or 4. However, this elbow is not distinct enough to be sure. Thus, the Silhouette Width Criterion (SWC) has been calculated. The SWC confirms that the optimal value of k is 3. This is interesting, as this value clearly indicates that the clusters are less likely to indicate a patient with heart disease than a normal patient. However, this could indicate subtypes within one of the targets or a selection of patients who are currently normal but heading towards heart disease (or vice versa). Thus, to confirm this clustering technique will be implemented and examined for this relationship.

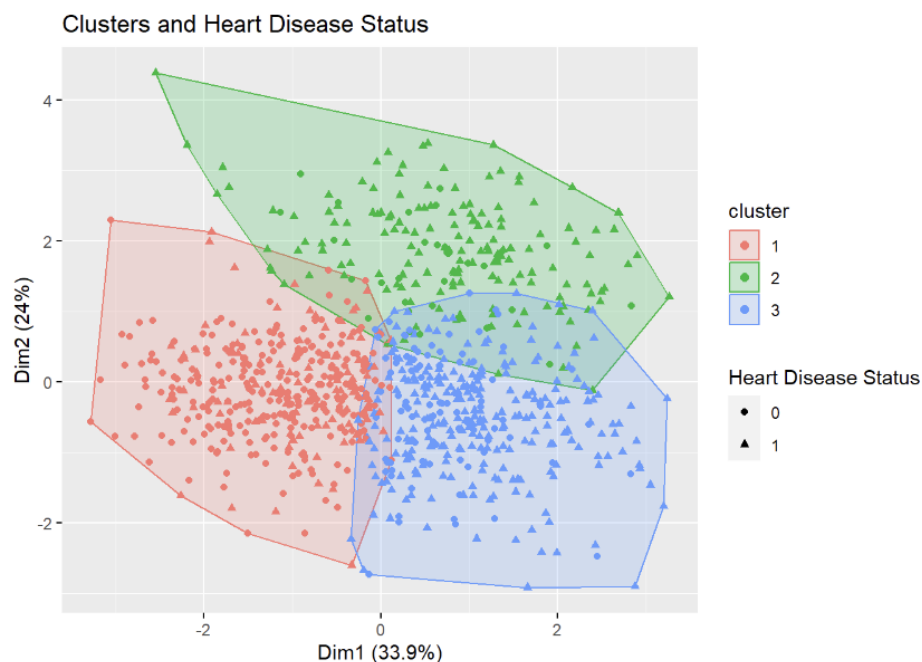


Figure 9 K-means Clustering Visualisation (clusters and heart disease status)

Examining the visualisation of the clustering (Figure 9), the k-means clustering appears to be effective. The visualisation shows three clusters with little overlap. Clusters 1 and 3 and Clusters 1 and 2 have minimal overlap, while Clusters 2 and 3 have more, though not enough to be considered significant.

However, these clusters do not describe the response variable (target, 0 = normal, 1 = heart disease). Examining the table of the cluster classification vs the target variable in Figure 10, approximately 30-40% of the observations belong to clusters other than that containing most of the observations with the same target value.

```
table(km.out$cluster, scaled.hd[[6]])
```

```
##
##      0    1
## 1 396 136
## 2  29 157
## 3 136 336
```

Figure 10 Tabular summary of clusters

In addition, the total variance of the dataset explained by the clustering can be determined by examining the results of the “Within cluster sum of squares by cluster” (Figure 11).

```
## Within cluster sum of squares by cluster:
## [1] 1320.7098  747.5514 1708.2481
## (between_SS / total_SS =  36.5 %)
```

Figure 11 Within cluster sum of squares by cluster output

This value is 36.5%, which implies that the clustering does not provide a significant explanation of the response variable and, thus, should not be used as a predictor of whether a patient has heart disease.

Conclusions

In conclusion, out of the four methodologies employed, the only method that did not provide a good explanation of the data is k-means clustering. Comparing the accuracy (Table 2) of the Naïve Bayes Classifier and the GLM and KNN models, all these models returned an accuracy between 0.7 and 0.9, the industry standard of a good model. Due to these models' accuracy, none is likely overfitted. However, out of all seven models, the GLM model using all features in the data set gave the best prediction of the response variable, though these values are subject to change with different partitioning of the training and test data.

Therefore, by using logistic regression, KNN classification or Naïve Bayes Classifier models, these features are sufficient for predicting whether a patient has heart disease within an acceptable level of accuracy, although based on the kappa statistic, the GLM models with all features, and significant features, which are the most accurate, prove to also be the most robust. Further analysis is needed to assess these models' accuracy on other patients.

Ideally, further analysis of other features not included in this dataset would be utilised to prove whether these are the best features at predicting heart disease. It would also be useful to assess which features are responsible for the clusters, as it is not whether heart disease is present or not, to draw further or new conclusions from this dataset.

References

- Abhigyan. (2020, June 14). *Importance of dimensionality reduction!!* Medium. Retrieved December 12, 2022, from <https://medium.com/analytics-vidhya/importance-of-dimensionality-reduction-d6a4c7289b92>
- FutureAnalytica. (2022, December 1). *Accuracy of a machine learning model*. Medium. Retrieved December 13, 2022, from <https://medium.com/@futureanalytica/accuracy-of-a-machine-learning-model-5d4f8e7dbfed>
- Heart Research Australia. (2021, March 4). *What is heart disease*. Heart Research Australia. Retrieved December 12, 2022, from <https://www.heartresearch.com.au/heart-disease/what-is-heart-disease/>
- Kaiser rule*. Displayr. (n.d.). Retrieved December 13, 2022, from https://docs.displayr.com/wiki/Kaiser_Rule
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159. <https://doi.org/10.2307/2529310>
- Providence. (2021, January 14). *Early heart disease detection saves lives*. PH&S OR. Retrieved December 12, 2022, from <https://www.providence.org/news/uf/644387305>
- RStudio Team (2022). RStudio: Integrated Development Environment for R. RStudio, PBC, Boston, MA URL <http://www.rstudio.com/>.
- Siddhartha, M. (2020, November 6). *Heart disease dataset (comprehensive)*. IEEE DataPort. Retrieved December 13, 2022, from <https://ieee-dataport.org/open-access/heart-disease-dataset-comprehensive>
- Using Horn's parallel analysis method in exploratory factor analysis for determining the number of factors. (2016). *Educational Sciences: Theory & Practice*. <https://doi.org/10.12738/estp.2016.2.0328>

Appendices

MA5810 Capstone Project- Using Machine Learning Methodologies to Predict Heart Disease

Samuel Bousfield

2022-12-13

1. LDA, QDA and/or Naive Bayes classification

Assumptions

LDA Assumptions:

- 1) response variable is categorical
- 2) predictor variables are normally distributed \
- 3) predictor variables have same variance

QDA Assumptions:

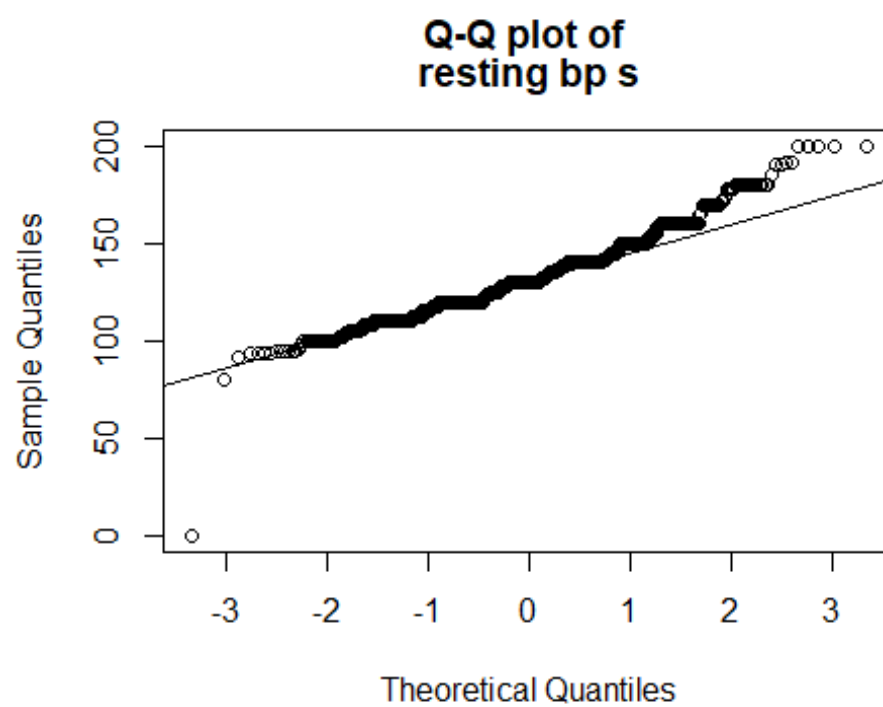
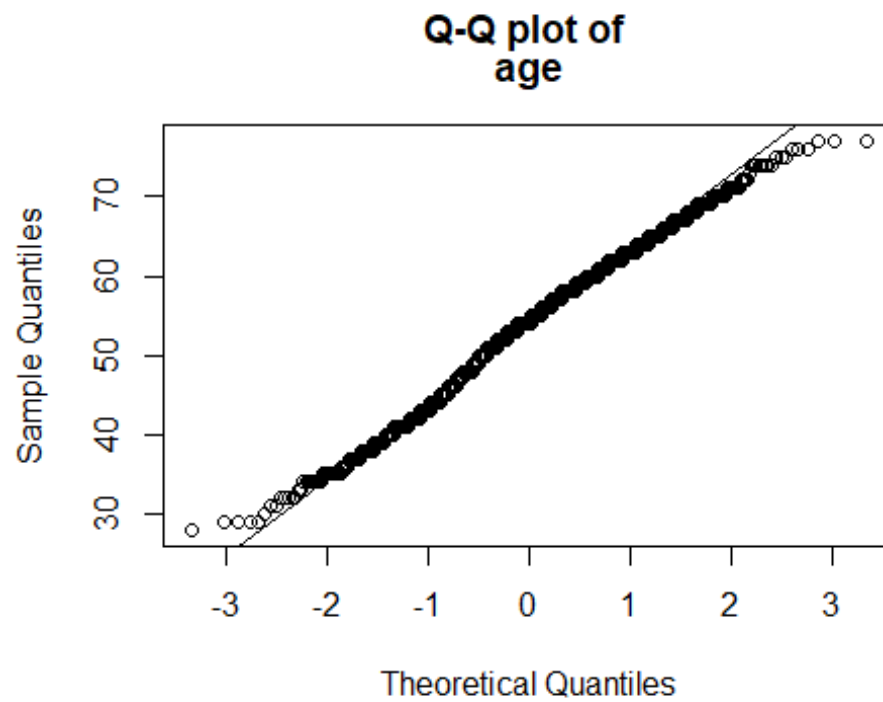
- 1) response variable is categorical
- 2) predictor variables are normally distributed

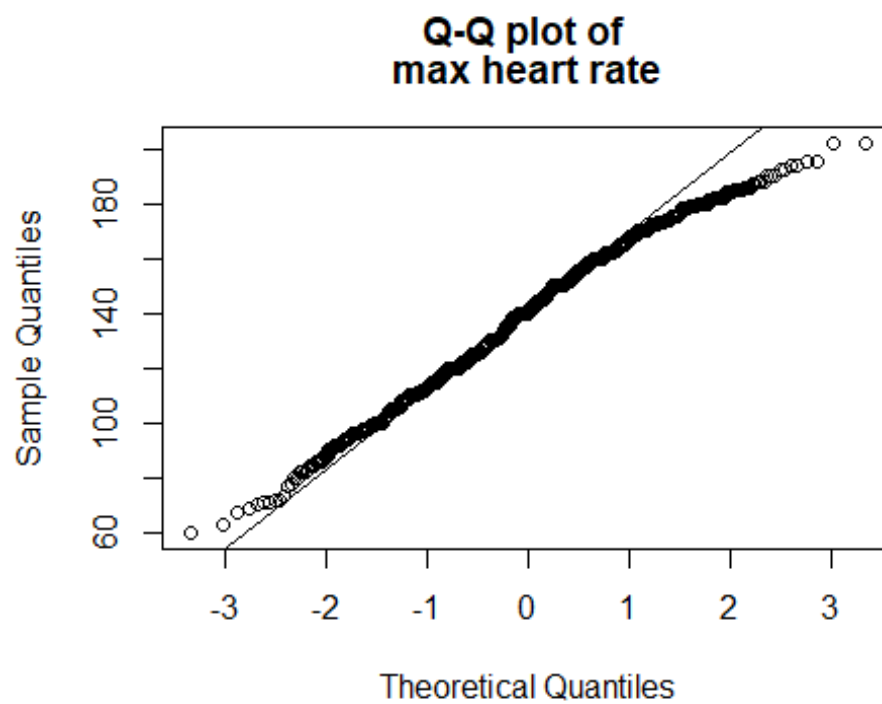
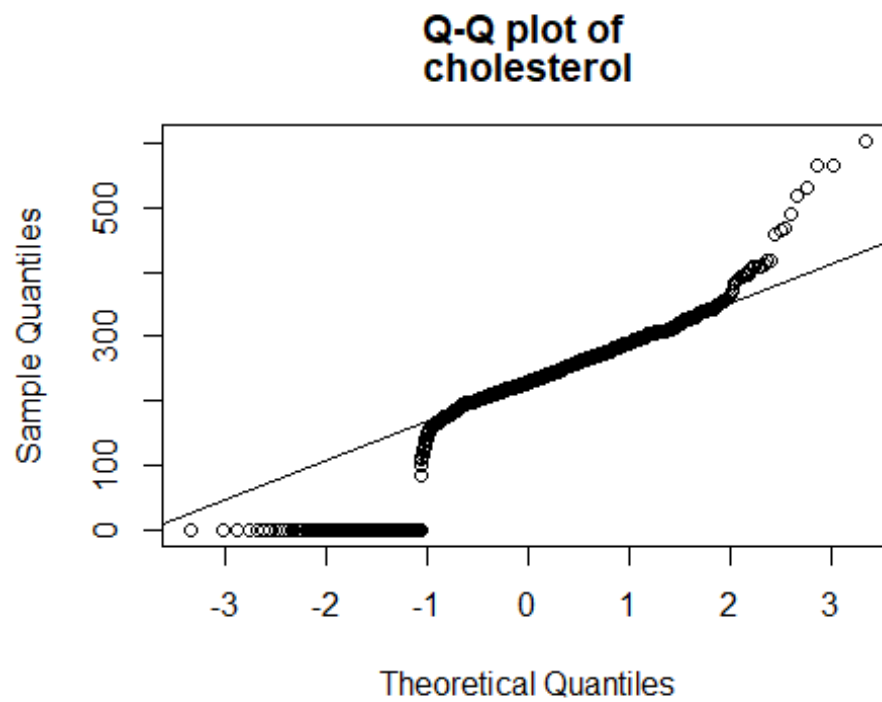
Assumption 1 - Categorical Response Variable

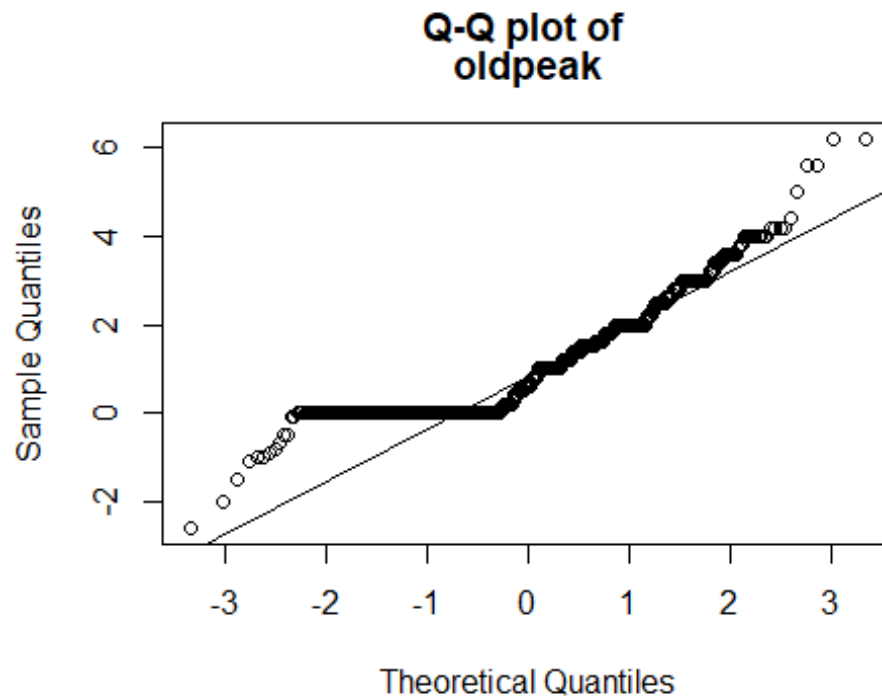
Response variable (target) is a binary category. Not all features are numeric, therefore to produce LDA or QDA, only some of the features can be used: Age, resting bp s, cholesterol, max heart rate and oldpeak.

Assumption 2 - Normality

```
#Make Q-Q Plots of all numeric features
numeric_features <- c("age", "resting bp s", "cholesterol", "max heart rate", "oldpeak")
for (feature in numeric_features){
  qqnorm(heart.disease[[feature]], main=c("Q-Q plot of ", feature))
  qqline(heart.disease[[feature]])
}
```







```
#Shapiro-wilk test
for (feature in numeric_features){
  x <- shapiro.test(heart.disease[[feature]])
  print(feature)
  print(x)
}
```

```
## [1] "age"
##
## Shapiro-Wilk normality test
##
## data: heart.disease[[feature]]
## W = 0.99126, p-value = 1.638e-06
##
## [1] "resting bp s"
##
## Shapiro-Wilk normality test
##
## data: heart.disease[[feature]]
## W = 0.96059, p-value < 2.2e-16
##
## [1] "cholesterol"
##
## Shapiro-Wilk normality test
##
## data: heart.disease[[feature]]
## W = 0.86594, p-value < 2.2e-16
##
## [1] "max heart rate"
##
## Shapiro-Wilk normality test
##
## data: heart.disease[[feature]]
## W = 0.98966, p-value = 1.981e-07
##
## [1] "oldpeak"
##
```

```
## Shapiro-Wilk normality test
##
## data: heart.disease[[feature]]
## W = 0.85902, p-value < 2.2e-16
```

These results prove that the data is not normally distributed. The plots show, These results reject the use of Linear Discriminant Analysis and Quadratic Discriminant Analysis on this dataset.

Assumption 3 - Equality of Variances

```
#use Bartlett test on numeric columns
bartlett.test(numeric.heart.disease[1:5])

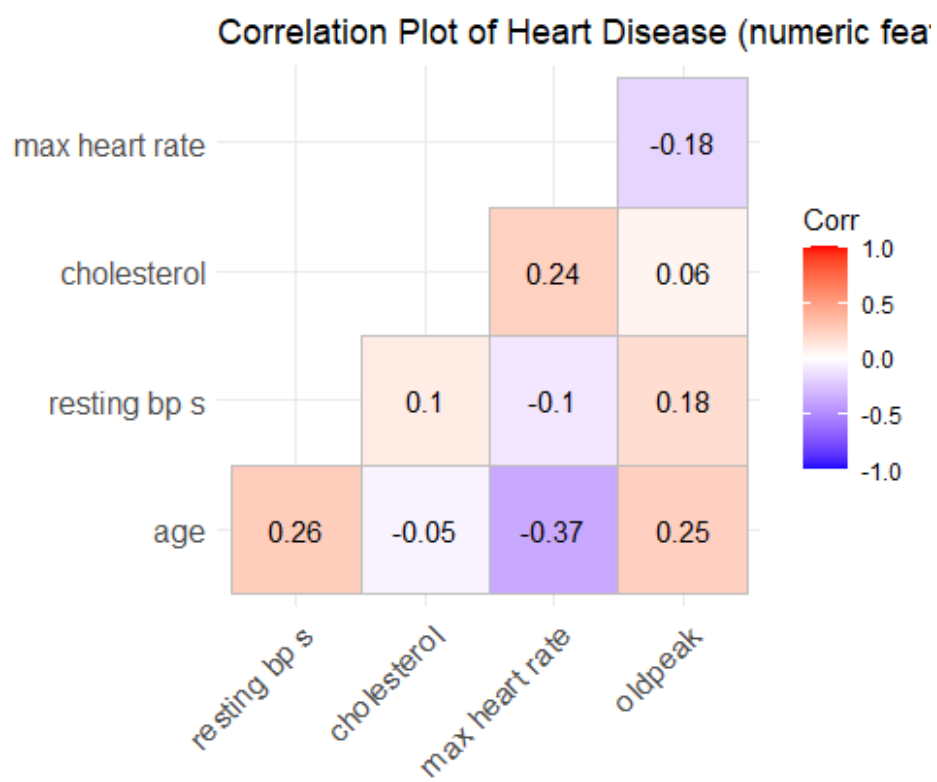
##
## Bartlett test of homogeneity of variances
##
## data: numeric.heart.disease[1:5]
## Bartlett's K-squared = 14818, df = 4, p-value < 2.2e-16
```

The results of the Bartlett test ($p < 2.2e-16$), rejects the null hypothesis (no difference in variances between groups), thus proving that the predictor variables do not have equal variances. Along with Assumption 2: Normality, this rejects the use of Linear Discriminant Analysis on this dataset.

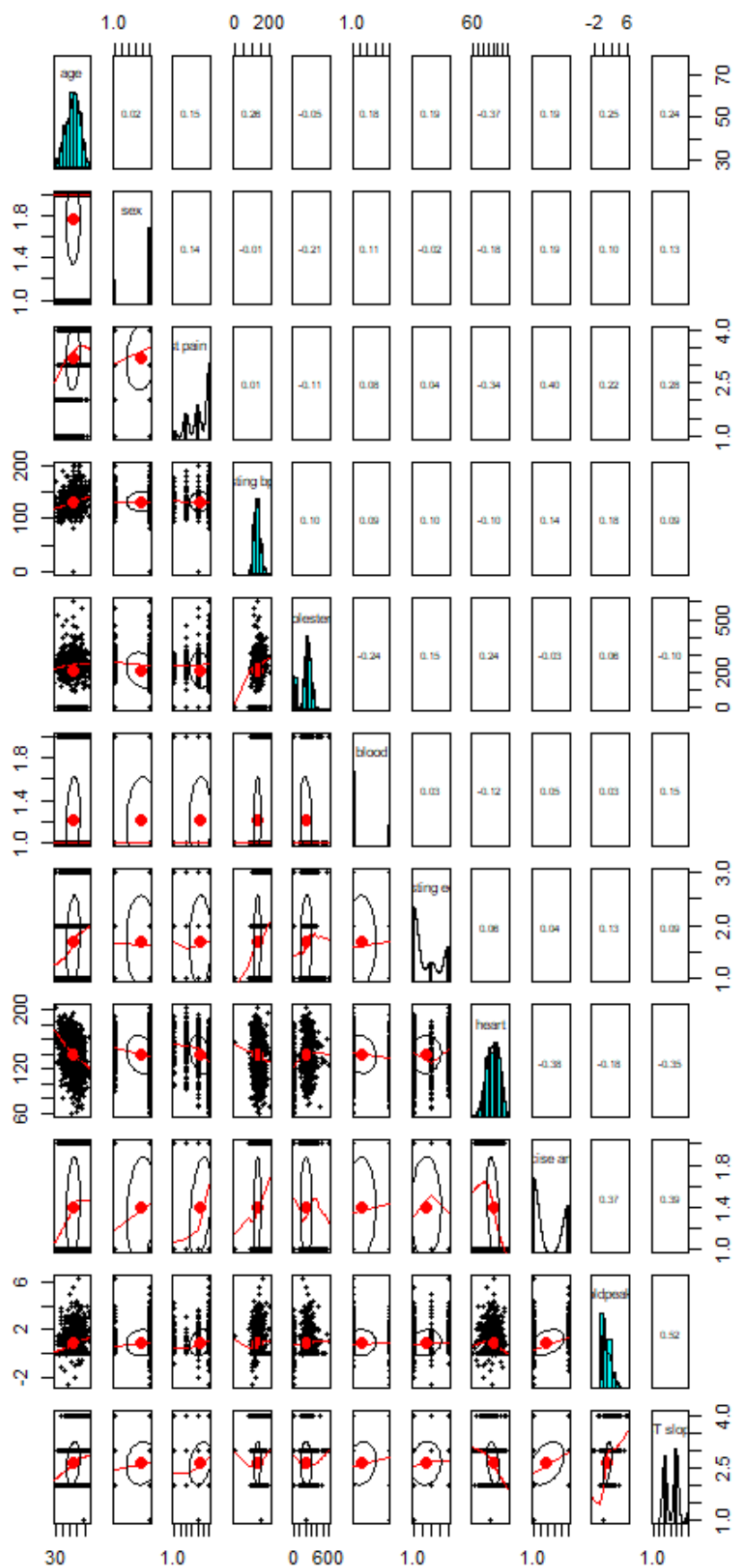
Naive Bayes Classifier (NBC) Assumptions :

1) Independence of features

```
#calculate and plot correlation matrix of numeric variables
corr_matrix <- cor(numeric.heart.disease[1:5])
ggcorrplot(corr_matrix, type = "lower", lab=TRUE, title = "Correlation Plot of Heart Disease (numeric features)")
```



```
#create panel pair correlation  
pairs.panels(heart.disease[1:11])
```



The correlation plot and pair plots show the max correlation coefficient is 0.52, between the “oldpeak” and “ST slope” variables. Anything above 0.5 or below -0.5 could be considered a relationship. Although a value this low is not conclusive evidence of non-independence of these variables, and the NBC typically exhibits good performance against violations of this assumption.

2) Equality of features

To perform a Naive Bayes Classifier, the assumption is that all variables have an equal effect on the outcome of the model. It can be assumed that these features will have an equal effect on the outcome.

Thus with both assumptions met, an NBC for this dataset will be calculated.

```
#Split data into training and test data. 80% training, 20% test
set.seed(1)
samples <- createDataPartition(numeric.heart.disease$target, p=0.8, list=FALSE)
train.heart.disease <- numeric.heart.disease[samples,]
test.heart.disease <- numeric.heart.disease[-samples,]

#implement classifier
NBC.heart.disease <- naiveBayes(target ~ ., data=train.heart.disease)
NBC.heart.disease

##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##      0      1
## 0.4711438 0.5288562
##
## Conditional probabilities:
##      age
## Y      [,1]      [,2]
## 0 50.77283 9.330828
## 1 56.04563 8.613406
##
##      resting bp s
## Y      [,1]      [,2]
## 0 129.2160 16.55794
## 1 133.8194 18.68865
##
##      cholesterol
## Y      [,1]      [,2]
## 0 234.0156 69.12652
## 1 192.0754 119.18697
##
##      max heart rate
## Y      [,1]      [,2]
## 0 150.3875 22.83337
## 1 130.0119 23.73084
##
##      oldpeak
## Y      [,1]      [,2]
## 0 0.4360802 0.7032379
## 1 1.3299603 1.2133597
```

```

#view confusion matrix on test data
NBC.conf <- confusionMatrix(predict(NBC.heart.disease, newdata=test.heart.disease), factor(
test.heart.disease$target))
print("Confusion Matrix - NBC Model")

## [1] "Confusion Matrix - NBC Model"

NBC.conf

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  0   1
##           0  85  26
##           1  27  99
##
##              Accuracy : 0.7764
##              95% CI : (0.7179, 0.8278)
##      No Information Rate : 0.5274
##      P-Value [Acc > NIR] : 1.928e-15
##
##              Kappa : 0.5512
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.7589
##              Specificity : 0.7920
##              Pos Pred Value : 0.7658
##              Neg Pred Value : 0.7857
##              Prevalence : 0.4726
##              Detection Rate : 0.3586
##      Detection Prevalence : 0.4684
##              Balanced Accuracy : 0.7755
##
##              'Positive' Class : 0
##

```

Viewing the confusion matrix for the application of the NBC against the test data, it is evident that this model provides good performance, with an accuracy of 0.7764, with industry standards being between 0.7 and 0.9. This value indicates the model does not suffer from over-fitting, and thus will be applicable to other data sets which utilise the same features. Examining the prediction results, and the sensitivity (true normal patient) and the specificity (true heart disease patient), the model is slightly better at predicting the presence of heart disease (0.7920) than 'normal' (0.7589).

It is important to note that these values will change depending on the specific partitioning of the training and test sets.

2. Logistic Regression classifiers and/or KNN for classification/regression

Given the strong performance

All Features Models

```

#split data into training and test data. 80% training, 20% test
set.seed(1)
samples <- createDataPartition(heart.disease$target, p=0.8, list=FALSE)
train.heart.disease <- heart.disease[samples,]
test.heart.disease <- heart.disease[-samples,]

```

```

#Log model training
glm.model <- train(form = `target` ~., data=train.heart.disease, method="glm")
summary(glm.model)

##
## Call:
## NULL
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.6072  -0.4204   0.1694   0.4787   2.6238
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    9.603201  535.413531   0.018  0.985690
## age            0.023896   0.012789   1.868  0.061701 .
## sex1          1.549821   0.258575   5.994  2.05e-09 ***
## `\\`chest pain type\\`2` -0.180069   0.466156  -0.386  0.699286
## `\\`chest pain type\\`3` -0.042665   0.416851  -0.102  0.918478
## `\\`chest pain type\\`4`  1.715610   0.410288   4.181  2.90e-05 ***
## `\\`resting bp s\\`    0.008330   0.006136   1.358  0.174602
## cholesterol    -0.004759   0.001158  -4.110  3.96e-05 ***
## `\\`fasting blood sugar\\`1` 0.841347   0.265950   3.164  0.001559 **
## `\\`resting ecg\\`1`    -0.058188   0.305023  -0.191  0.848709
## `\\`resting ecg\\`2`     0.231446   0.240996   0.960  0.336868
## `\\`max heart rate\\`    -0.002515   0.004849  -0.519  0.604093
## `\\`exercise angina\\`1`  0.775874   0.234269   3.312  0.000927 ***
## oldpeak        0.416158   0.111162   3.744  0.000181 ***
## `\\`ST slope\\`1`      -14.523779  535.411310  -0.027  0.978359
## `\\`ST slope\\`2`      -12.339176  535.411321  -0.023  0.981613
## `\\`ST slope\\`3`      -13.662666  535.411450  -0.026  0.979642
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1317.96  on 952  degrees of freedom
## Residual deviance:  653.01  on 936  degrees of freedom
## AIC: 687.01
##
## Number of Fisher Scoring iterations: 12

glm.model

## Generalized Linear Model
##
## 953 samples
## 11 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 953, 953, 953, 953, 953, 953, ...
## Resampling results:
##
##   Accuracy   Kappa
##  0.8385123  0.6746585

#knn model training
knn.model <- train(form=`target`~., data=train.heart.disease, method="knn")
summary(knn.model)

##           Length Class      Mode
## learn           2    -none-    list
## k                1    -none-    numeric
## theDots          0    -none-    list

```

```

## xNames      16      -none-      character
## problemType 1      -none-      character
## tuneValue   1      data.frame list
## obsLevels   2      -none-      character
## param       0      -none-      list

knn.model

## k-Nearest Neighbors
##
## 953 samples
## 11 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 953, 953, 953, 953, 953, 953, ...
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 5 0.6859582 0.3692459
## 7 0.6853211 0.3680952
## 9 0.6953160 0.3882514
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 9.

#calculate confusion matrices of model on test data
glm.conf <- confusionMatrix(predict(glm.model, newdata=test.heart.disease), factor(test.heart.disease$target))
knn.conf <- confusionMatrix(predict(knn.model, newdata=test.heart.disease), factor(test.heart.disease$target))

#display confusion matrices of model vs test data
print("Confusion Matrix - GLM Model")

## [1] "Confusion Matrix - GLM Model"

glm.conf

## Confusion Matrix and Statistics
##
##              Reference
## Prediction  0    1
##      0    91   16
##      1    21  109
##
##              Accuracy : 0.8439
##              95% CI : (0.7913, 0.8876)
##      No Information Rate : 0.5274
##      P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.6861
##
##      McNemar's Test P-Value : 0.5108
##
##              Sensitivity : 0.8125
##              Specificity : 0.8720
##      Pos Pred Value : 0.8505
##      Neg Pred Value : 0.8385
##      Prevalence : 0.4726
##      Detection Rate : 0.3840
##      Detection Prevalence : 0.4515
##      Balanced Accuracy : 0.8422
##

```

```
##      'Positive' Class : 0
##

print("Confusion Matrix - KNN Model")

## [1] "Confusion Matrix - KNN Model"

knn.conf

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0   1
##           0  75  33
##           1  37  92
##
##           Accuracy : 0.7046
##           95% CI : (0.6421, 0.7619)
##           No Information Rate : 0.5274
##           P-Value [Acc > NIR] : 1.986e-08
##
##           Kappa : 0.4064
##
##  Mcnemar's Test P-Value : 0.7199
##
##           Sensitivity : 0.6696
##           Specificity : 0.7360
##           Pos Pred Value : 0.6944
##           Neg Pred Value : 0.7132
##           Prevalence : 0.4726
##           Detection Rate : 0.3165
##           Detection Prevalence : 0.4557
##           Balanced Accuracy : 0.7028
##
##      'Positive' Class : 0
##
```

Examining the results of the glm model, it is evident that not all features are as important in determining the outcome. To further improve model accuracy, only the features with a p-value of <0.05 will be used. These features are “sex”, “chest pain type”, “cholesterol”, “fasting blood sugar”, “exercise angina” and “oldpeak”. The high p-values in the ST slope variable indicate multicollinearity and thus is also likely to be a significant feature.

Significant Features Models

```
#split data into training and test data. 80% training, 20% test
set.seed(1)
samples <- createDataPartition(heart.disease$target, p=0.8, list=FALSE)
train.heart.disease <- heart.disease[samples,]
test.heart.disease <- heart.disease[-samples,]

#train models
glm.model.sig <- train(form = `target`~`sex`+`chest pain type`+`cholesterol`+`fasting blood
sugar`+`exercise angina`+`oldpeak`+`ST slope`, data=train.heart.disease, method="glm")
summary(glm.model.sig)

##
## Call:
## NULL
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.7738  -0.4219   0.1785   0.4634   2.7319
##
## Coefficients:
```

```
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)    12.153461  535.411540   0.023 0.981890
## sex1           1.495438   0.254810   5.869 4.39e-09 ***
## `\\`chest pain type\\`2` -0.318861   0.456491  -0.699 0.484861
## `\\`chest pain type\\`3` -0.143196   0.407533  -0.351 0.725309
## `\\`chest pain type\\`4`  1.625650   0.395542   4.110 3.96e-05 ***
## cholesterol    -0.004407   0.001085  -4.061 4.89e-05 ***
## `\\`fasting blood sugar\\`1`  0.919708   0.261561   3.516 0.000438 ***
## `\\`exercise angina\\`1`    0.846917   0.225068   3.763 0.000168 ***
## oldpeak        0.483927   0.107921   4.484 7.32e-06 ***
## `\\`ST slope\\`1`          -15.055969  535.411260  -0.028 0.977566
## `\\`ST slope\\`2`          -12.831152  535.411265  -0.024 0.980880
## `\\`ST slope\\`3`          -14.119177  535.411391  -0.026 0.978962
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 1318.0  on 952  degrees of freedom
## Residual deviance:  663.4  on 941  degrees of freedom
## AIC: 687.4
##
## Number of Fisher Scoring iterations: 12

glm.model.sig

## Generalized Linear Model
##
## 953 samples
## 7 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 953, 953, 953, 953, 953, 953, ...
## Resampling results:
##
##   Accuracy   Kappa
##  0.8425938  0.6830725

knn.model.sig <-train(form = `target`~`sex`+`chest pain type`+`cholesterol`+`fasting blood
sugar`+`exercise angina`+`oldpeak`+`ST slope`, data=train.heart.disease, method="knn")
summary(knn.model.sig)

##           Length Class      Mode
## learn         2    -none-    list
## k              1    -none-    numeric
## theDots        0    -none-    list
## xNames        11    -none-    character
## problemType    1    -none-    character
## tuneValue      1    data.frame list
## obsLevels      2    -none-    character
## param          0    -none-    list

knn.model.sig

## k-Nearest Neighbors
##
## 953 samples
## 7 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 953, 953, 953, 953, 953, 953, ...
## Resampling results across tuning parameters:
```

```
##
## k Accuracy Kappa
## 5 0.7450952 0.4891199
## 7 0.7454539 0.4899978
## 9 0.7375319 0.4741416
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 7.

#compute then display confusion matrices of models vs test data
glm.sig.conf <- confusionMatrix(predict(glm.model.sig, newdata=test.heart.disease), factor(
test.heart.disease$target))
knn.sig.conf <- confusionMatrix(predict(knn.model.sig, newdata=test.heart.disease), factor(
test.heart.disease$target))

print("Confusion Matrix - Significant Features GLM Model")

## [1] "Confusion Matrix - Significant Features GLM Model"

glm.sig.conf

## Confusion Matrix and Statistics
##
##          Reference
## Prediction  0    1
##          0  90  17
##          1  22 108
##
##          Accuracy : 0.8354
##          95% CI : (0.782, 0.8803)
##    No Information Rate : 0.5274
##    P-Value [Acc > NIR] : <2e-16
##
##          Kappa : 0.6691
##
##  McNemar's Test P-Value : 0.5218
##
##          Sensitivity : 0.8036
##          Specificity : 0.8640
##          Pos Pred Value : 0.8411
##          Neg Pred Value : 0.8308
##          Prevalence : 0.4726
##          Detection Rate : 0.3797
##          Detection Prevalence : 0.4515
##          Balanced Accuracy : 0.8338
##
##          'Positive' Class : 0
##

print("Confusion Matrix - Significant Features KNN Model")

## [1] "Confusion Matrix - Significant Features KNN Model"

knn.sig.conf

## Confusion Matrix and Statistics
##
##          Reference
## Prediction  0    1
##          0  76  28
##          1  36  97
##
##          Accuracy : 0.73
##          95% CI : (0.6687, 0.7854)
##    No Information Rate : 0.5274
##    P-Value [Acc > NIR] : 1.345e-10
```

```
##
##           Kappa : 0.4563
##
## Mcnemar's Test P-Value : 0.3816
##
##           Sensitivity : 0.6786
##           Specificity : 0.7760
##           Pos Pred Value : 0.7308
##           Neg Pred Value : 0.7293
##           Prevalence : 0.4726
##           Detection Rate : 0.3207
##           Detection Prevalence : 0.4388
##           Balanced Accuracy : 0.7273
##
##           'Positive' Class : 0
##
```

These new models, only utilising the significant features from the original glm model, have changed the models accuracy. In the case of the glm models, the accuracy has slightly decreased from 0.8439 on the original model to 0.8354 in the new model.

However, the KNN model has increased in accuracy. In the original model the accuracy is 0.7046, in the model using only significant features the accuracy is 0.7468.

Comparing the confusion matrices where the model is applied to the testing data, in the case of the knn model, the accuracy increases from 0.7046 in the original model, to 0.7468 in the model containing only the significant features. Likewise, there is an increase in the glm model accuracy, from 0.8101 to 0.8228. However, whilst there is an increase in the accuracy of both models when only the significant ($p < 0.05$) features are used. The knn model is clearly the lesser of the two when predicting the presence of heart disease.

As with the NBC model, it is important to note that these values will change depending on the specific partitioning of the training and test sets.

3. Principal Component Analysis (PCA)

As the glm model was the better of the two (glm vs knn), and the glm model using only significant features was the best of the four models examined, this be the one carried forward. In an attempt to provide a reduction to the dimensionality in this model, PCA will be utilised.

Principal Component Generation

```
#Calculate Principal Components
PCA.hd <- prcomp(numeric.heart.disease[1:5], scale = TRUE)
#Reverse signs
PCA.hd$rotation <- PCA.hd$rotation*-1
#view Principal components
PCA.hd$rotation

##           PC1          PC2          PC3          PC4          PC5
## age          -0.5931435  0.01872862 -0.12889588 -0.4554649 -0.65097431
## resting bp s -0.3913147  0.44516375 -0.64169873  0.4522453  0.17999716
## cholesterol  0.1306225  0.75789321  0.12494725 -0.5643215  0.27288298
## max heart rate 0.5337018  0.38870933 -0.03082717  0.3078181 -0.68437207
## oldpeak      -0.4394926  0.27564727  0.74501440  0.4181091 -0.03167348

#reverse signs of scores
PCA.hd$x <- PCA.hd$x*-1
```

```

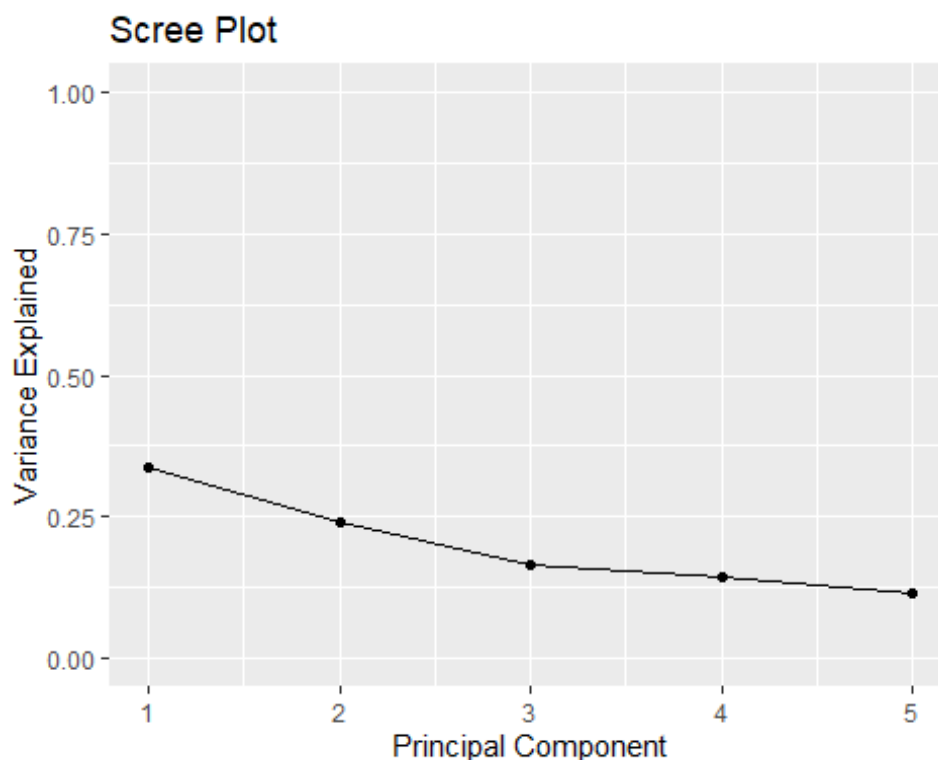
#view head of scores
#head(PCA.hd) #Commented out to prevent output in RMarkdown file

#calculate total variance explained by each principal component
explained_variance <- PCA.hd$sdev^2/ sum(PCA.hd$sdev^2)
explained_variance

## [1] 0.3387147 0.2399997 0.1646287 0.1421444 0.1145125

#scree plot
qplot(c(1:5), explained_variance) +
  geom_line() +
  xlab("Principal Component") +
  ylab("Variance Explained") +
  ggtitle("Scree Plot") +
  ylim(0, 1)

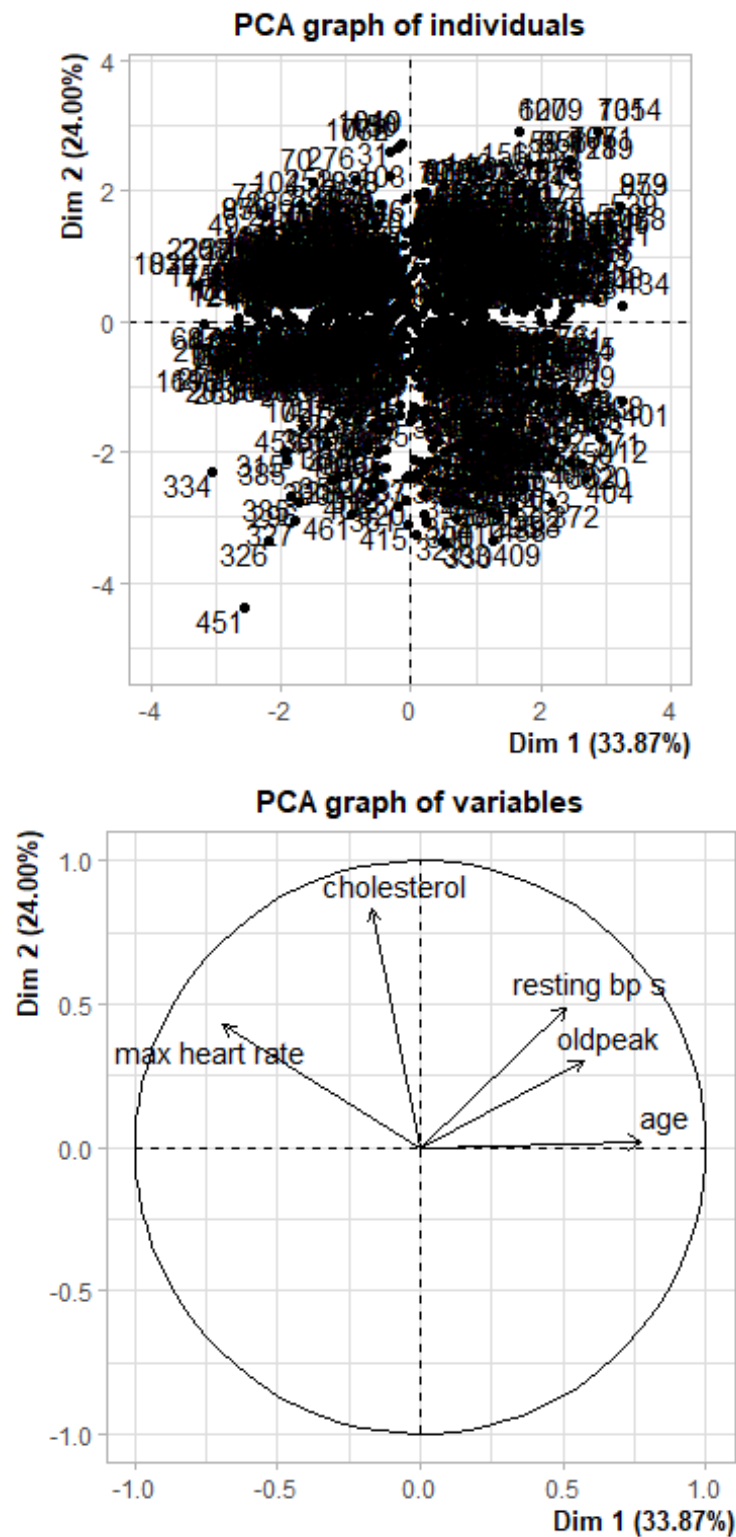
```



Examining the scree plot, and explained variance it is evident that there is no clear “elbow” at which to choose the number of Principal Components. Therefore, the Principal Components will be examined according to the Kaiser Rule and Horn’s Parallel Analysis. The Kaiser rule examines the Principal components will be examined for those with an eigenvalue of >1 , which indicates a Principal Component that performs better than an individual variable. Horn’s parallel analysis compares the generated eigenvalues to a simulated set of eigenvalues, to determine the number of Principal Components to keep.

Principal Component Selection

```
PCA.hd2 <- PCA(numeric.heart.disease[1:5])
```



```
summary(PCA.hd2, ncp=4)
```

```
##
## Call:
## PCA(X = numeric.heart.disease[1:5])
##
##
## Eigenvalues
##
```

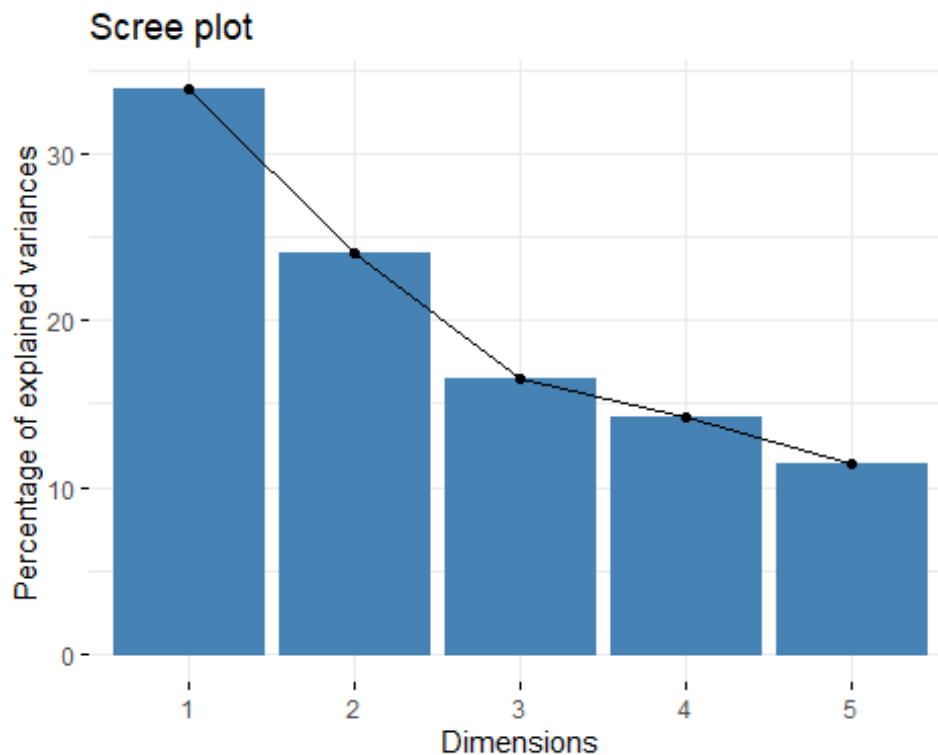
	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
## Variance	1.694	1.200	0.823	0.711	0.573

```

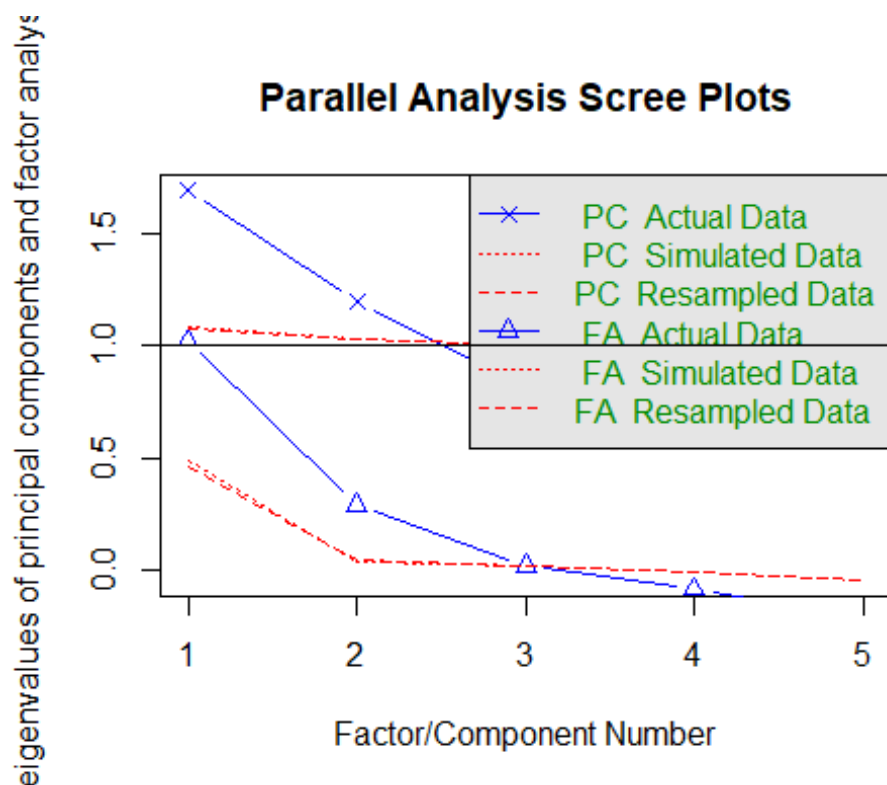
## % of var.          33.871  24.000  16.463  14.214  11.451
## Cumulative % of var. 33.871  57.871  74.334  88.549 100.000
##
## Individuals (the 10 first)
##
##          Dist   Dim.1   ctr   cos2   Dim.2   ctr   cos2   Dim.3
## 1          2.293 | -1.853  0.170  0.653 |  1.008  0.071  0.193 | -0.660
## 2          1.748 |  0.024  0.000  0.000 |  0.706  0.035  0.163 | -0.912
## 3          2.669 | -0.700  0.024  0.069 | -0.413  0.012  0.024 | -0.187
## 4          1.519 |  0.655  0.021  0.186 | -0.180  0.002  0.014 |  0.313
## 5          1.474 |  0.415  0.009  0.079 | -0.186  0.002  0.016 | -1.258
## 6          2.580 | -2.365  0.278  0.840 |  0.865  0.052  0.112 |  0.116
## 7          1.756 | -1.640  0.133  0.873 |  0.356  0.009  0.041 | -0.441
## 8          1.479 | -0.872  0.038  0.348 | -0.754  0.040  0.260 |  0.132
## 9          1.951 | -0.451  0.010  0.054 |  0.130  0.001  0.004 |  0.360
## 10         1.631 | -0.677  0.023  0.172 | -0.291  0.006  0.032 | -0.015
##
##          ctr   cos2   Dim.4   ctr   cos2
## 1          0.045  0.083 |  0.458  0.025  0.040 |
## 2          0.085  0.272 |  1.311  0.203  0.562 |
## 3          0.004  0.005 | -0.502  0.030  0.035 |
## 4          0.010  0.043 |  0.242  0.007  0.025 |
## 5          0.162  0.728 | -0.058  0.000  0.002 |
## 6          0.001  0.002 | -0.289  0.010  0.013 |
## 7          0.020  0.063 |  0.233  0.006  0.018 |
## 8          0.002  0.008 | -0.874  0.090  0.349 |
## 9          0.013  0.034 |  1.131  0.151  0.336 |
## 10         0.000  0.000 | -1.024  0.124  0.394 |
##
## Variables
##
##          Dim.1   ctr   cos2   Dim.2   ctr   cos2   Dim.3   ctr
## age          0.772 35.182  0.596 |  0.021  0.035  0.000 | -0.117  1.661
## resting bp s |  0.509 15.313  0.259 |  0.488 19.817  0.238 | -0.582 41.178
## cholesterol | -0.170  1.706  0.029 |  0.830 57.440  0.689 |  0.113  1.561
## max heart rate | -0.695 28.484  0.482 |  0.426 15.109  0.181 | -0.028  0.095
## oldpeak      |  0.572 19.315  0.327 |  0.302  7.598  0.091 |  0.676 55.505
##
##          cos2   Dim.4   ctr   cos2
## age          0.014 | -0.384 20.745  0.147 |
## resting bp s |  0.339 |  0.381 20.453  0.145 |
## cholesterol |  0.013 | -0.476 31.846  0.226 |
## max heart rate | 0.001 |  0.260  9.475  0.067 |
## oldpeak      |  0.457 |  0.352 17.482  0.124 |

fviz_screplot(PCA.hd2)

```



```
para.heart.disease <- fa.parallel(numeric.heart.disease[1:5])
```



```
## Parallel analysis suggests that the number of factors = 2 and the number of components = 2
```

```
para.heart.disease
```

```
## Call: fa.parallel(x = numeric.heart.disease[1:5])
```

```
## Parallel analysis suggests that the number of factors = 2 and the number of components
```

```
= 2
##
## Eigen Values of
## Original factors Resampled data Simulated data Original components
## 1 1.03 0.46 0.48 1.69
## 2 0.29 0.04 0.04 1.20
## Resampled components Simulated components
## 1 1.08 1.09
## 2 1.03 1.03
```

The results from both the Kaiser Rule and Horn's Parallel Analysis indicate that two Principal Components should be kept. Therefore the GLM and KNN models will be generated using two Principal Components.

PCA Models

```
set.seed(1)
#bind PCA columns to dataset
PC.heart.disease <- cbind(heart.disease, PCA.hd$x[,1:5])
#split data into training (80%) and test (20%) datasets
samples <- createDataPartition(PC.heart.disease$target, p=0.8, list=FALSE)
train.PC <- PC.heart.disease[samples,]
test.PC <- PC.heart.disease[-samples,]

#test models on Principal Components
glm.model.pca <- train(form = `target`~`PC1`+`PC2`, data=train.PC, method="glm")
glm.model.pca

## Generalized Linear Model
##
## 953 samples
## 2 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 953, 953, 953, 953, 953, 953, ...
## Resampling results:
##
## Accuracy Kappa
## 0.7189079 0.4360356

knn.model.pca <- train(form = `target`~`PC1`+`PC2`, data=train.PC, method="knn")
knn.model.pca

## k-Nearest Neighbors
##
## 953 samples
## 2 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 953, 953, 953, 953, 953, 953, ...
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 5 0.6960008 0.3895182
## 7 0.7058797 0.4096497
## 9 0.7121441 0.4220567
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 9.
```

```

#compute and display confusion matrices of models vs test data
glm.pca.conf <- confusionMatrix(predict(glm.model.pca, newdata=test.PC), factor(test.PC$target))
knn.pca.conf <- confusionMatrix(predict(knn.model.pca, newdata=test.PC), factor(test.PC$target))
print("Confusion Matrix - Principal Component GLM Model")

## [1] "Confusion Matrix - Principal Component GLM Model"

glm.pca.conf

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0   1
##           0  72  30
##           1  40  95
##
##               Accuracy : 0.7046
##               95% CI : (0.6421, 0.7619)
##           No Information Rate : 0.5274
##           P-Value [Acc > NIR] : 1.986e-08
##
##               Kappa : 0.4047
##
##  McNemar's Test P-Value : 0.2821
##
##           Sensitivity : 0.6429
##           Specificity : 0.7600
##           Pos Pred Value : 0.7059
##           Neg Pred Value : 0.7037
##           Prevalence : 0.4726
##           Detection Rate : 0.3038
##           Detection Prevalence : 0.4304
##           Balanced Accuracy : 0.7014
##
##           'Positive' Class : 0
##

print("Confusion Matrix - Principal Component KNN Model")

## [1] "Confusion Matrix - Principal Component KNN Model"

knn.pca.conf

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0   1
##           0  75  29
##           1  37  96
##
##               Accuracy : 0.7215
##               95% CI : (0.6598, 0.7776)
##           No Information Rate : 0.5274
##           P-Value [Acc > NIR] : 7.713e-10
##
##               Kappa : 0.4393
##
##  McNemar's Test P-Value : 0.3889
##
##           Sensitivity : 0.6696
##           Specificity : 0.7680
##           Pos Pred Value : 0.7212
##           Neg Pred Value : 0.7218
##           Prevalence : 0.4726

```

```
##      Detection Rate : 0.3165
##      Detection Prevalence : 0.4388
##      Balanced Accuracy : 0.7188
##
##      'Positive' Class : 0
##
```

The output of the confusion matrices using Principal Components on the test data, the accuracy of the GLM model is 0.7046, and the accuracy of the KNN model is 0.7215. This is somewhat expected, as the GLM algorithm does well with high dimensionality and the KNN algorithm does well with low dimensionality. Thus when compared to the models with all features and significant features, the GLM models did better than the KNN models, but when the number of features was reduced using PCA, the KNN model did better than the GLM model.

However it is important to note, that the difference in accuracy is only 0.0169 (1.69%), and both models are greater than 70% accurate. This means that both models using Principal Components provide good accuracy according to industry standards. It is also important to note that the exact values that this model provides may change with a different partition of the training and test data.

```
features_used <- c("All", "All", "All", "Sig Features", "Sig Features", "PC1 and PC2", "PC1
and PC2")
methodology <- c("NBC", "GLM", "KNN", "GLM", "KNN", "GLM", "KNN")
accuracy <- c(NBC.conf[["overall"]][["Accuracy"]], glm.conf[["overall"]][["Accuracy"]], knn
.conf[["overall"]][["Accuracy"]], glm.sig.conf[["overall"]][["Accuracy"]], knn.sig.conf[["o
verall"]][["Accuracy"]], glm.pca.conf[["overall"]][["Accuracy"]], knn.pca.conf[["overall"]
][["Accuracy"]])
kappa <- c(NBC.conf[["overall"]][["Kappa"]], glm.conf[["overall"]][["Kappa"]], knn.conf[["o
verall"]][["Kappa"]], glm.sig.conf[["overall"]][["Kappa"]], knn.sig.conf[["overall"]][["Kap
pa"]], glm.pca.conf[["overall"]][["Kappa"]], knn.pca.conf[["overall"]][["Kappa"]])

test.accuracy <- data.frame(features_used, methodology, accuracy, kappa)
test.accuracy
```

##	features_used	methodology	accuracy	kappa
## 1	All	NBC	0.7763713	0.5511845
## 2	All	GLM	0.8438819	0.6860927
## 3	All	KNN	0.7046414	0.4063976
## 4	Sig Features	GLM	0.8354430	0.6691248
## 5	Sig Features	KNN	0.7299578	0.4562661
## 6	PC1 and PC2	GLM	0.7046414	0.4047363
## 7	PC1 and PC2	KNN	0.7215190	0.4392744

4. Cluster Analysis

With the use of the Naive Bayes Classifier, and the classification/regression models (glm/knn, glm/knn with significant features, glm/knn with Principal Components), the goal was to examine the data to discover a classification methodology of whether a patient has heart disease or not, and this somewhat succeeded, the best model, glm with significant features returned a model that classified patients with approximately 82% accuracy. Interestingly, while the data contains a binary classification (normal or heart disease), the knn models found further clusters within the data. In the original model, and PC models $k = 9$, and in the significant figures model, $k=5$.

It is important for the best patient outcomes to discover whether these further clusters indicate differences in normal indicators vs differences in heart disease indicators or

whether these clusters could be indicative of a state where heart disease is not yet present, but is likely.

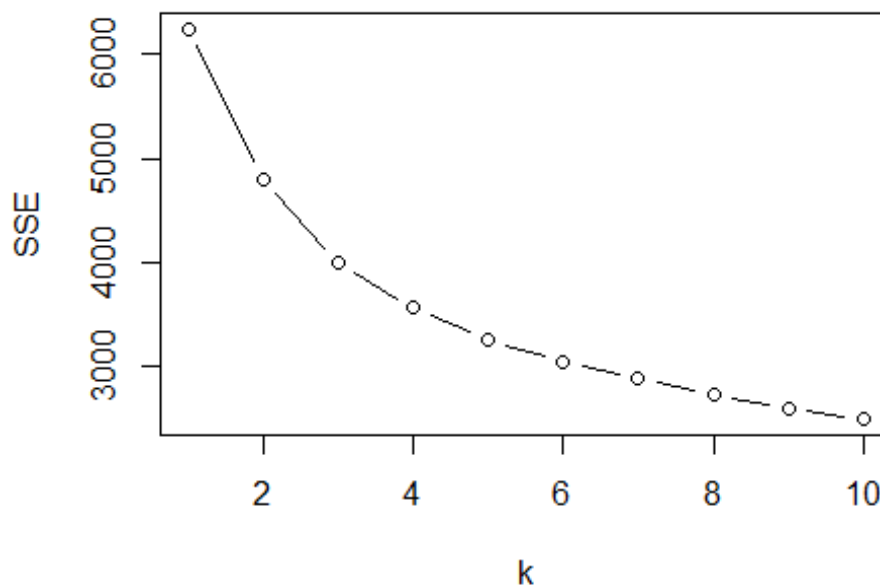
Scale Data

```
#scale data
scaled.hd <- data.frame(scale(numeric.heart.disease[1:5]))
scaled.hd <- cbind(scaled.hd, heart.disease$target)
names(scaled.hd)[names(scaled.hd)=="heart.disease$target"] <- "target"
```

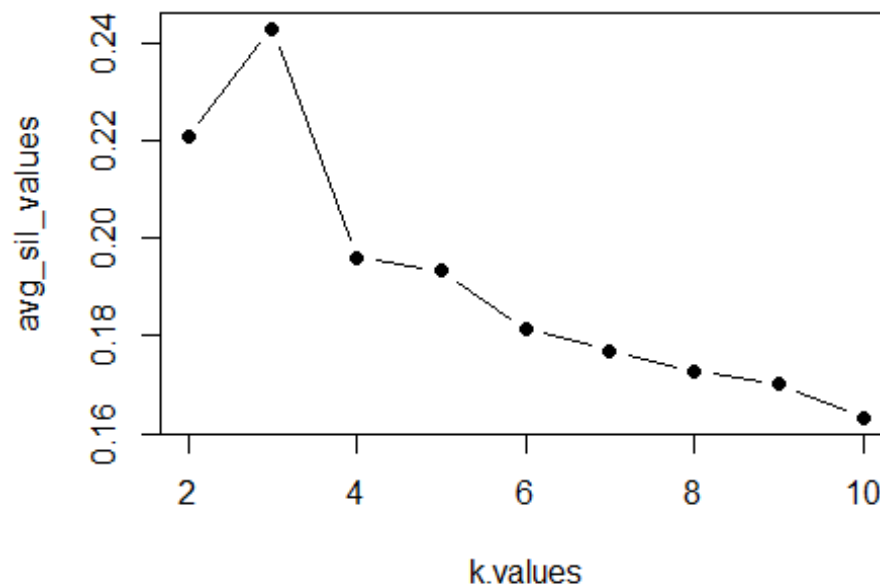
The data must be scaled initially. This is because k-means clustering is a distance based algorithm, and is thus biased towards variables with higher magnitudes. Scaling the data prevents this by ensuring all variables have the same magnitude.

Optimal k value

```
#SSE
set.seed(0)
SSE <- rep(0, 10)
for (k in 1:10){
  km.out <- kmeans(x = scaled.hd, centers = k, nstart = 10)
  SSE[k] <- km.out$tot.withinss
}
plot(1:10, SSE, xlab="k", ylab="SSE", type = "b")
```



```
#SWC to get optimal k
set.seed(1)
k.values <- 2:10
avg_sil <- function(k){
  km.res <- kmeans(scaled.hd, centers = k, nstart = 25)
  ss <- silhouette(km.res$cluster, dist(scaled.hd))
  mean(ss[,3])
}
avg_sil_values <- map_dbl(k.values, avg_sil)
plot(k.values, avg_sil_values, type = "b", pch = 19)
```



An examination for the elbow of the SSE graph appears to be at either 3, or 4. However, this elbow is not distinct enough to be certain. Thus, the Silhouette Width Criterion (SWC) has been calculated. The SWC confirms the optimal value of k is 3. This is interesting, as this value clearly indicates that the clusters are not likely to be indicative of a patient with heart disease vs a normal patient. However, this could indicate subtypes within one of the targets, or a selection of patients who are currently normal, but heading towards heart disease (or vice versa). Thus to confirm this clustering technique will be implemented and examined for this relationship.

K-means Clustering Implementation

```
#split data into training and test data. 80% training, 20% test
#set.seed(1)
#samples <- createDataPartition(scaled.hd$target, p=0.8, List=FALSE)
#train.scaled <- scaled.hd[samples,]
#test.scaled <- scaled.hd[-samples,]

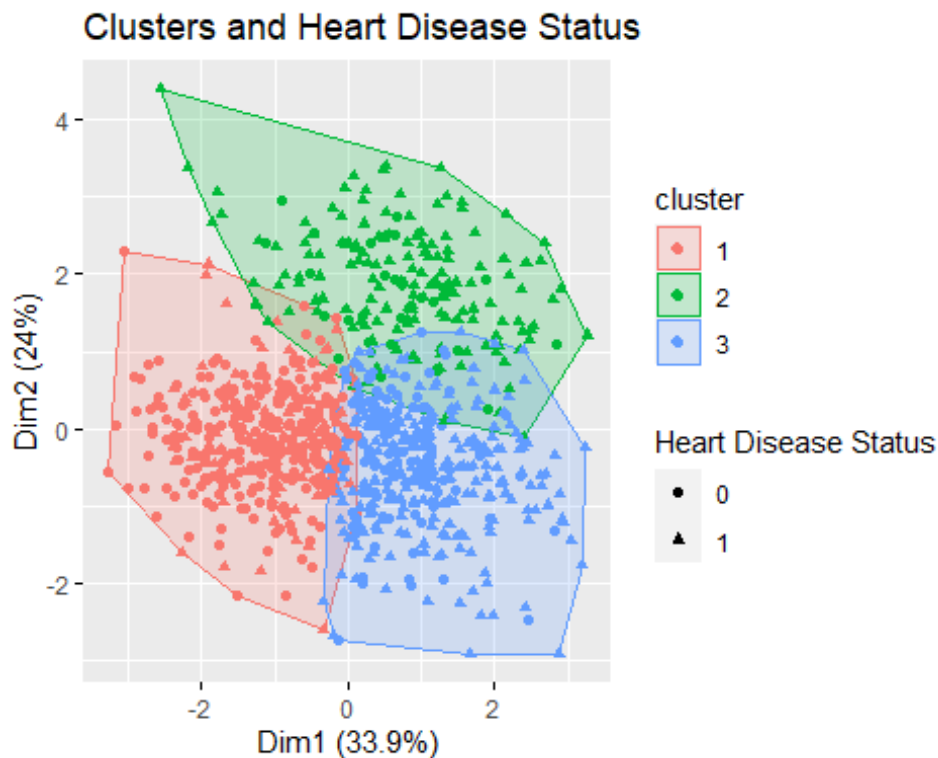
km.out <- kmeans(x = scaled.hd[1:5], centers = 3, nstart = 20)
hd.clustored <- data.frame(scaled.hd, cluster=factor(km.out$cluster))
km.out

## K-means clustering with 3 clusters of sizes 532, 186, 472
##
## Cluster means:
##      age resting.bp.s cholesterol max.heart.rate    oldpeak
## 1 -0.6825521  -0.3257002   0.2541237    0.6604724 -0.4929915
## 2  0.3280710  -0.1342279  -1.9223533   -0.7789881 -0.1159851
## 3  0.6400350   0.4199976   0.4711100   -0.4374566  0.6013659
##
## Clustering vector:
##      [1] 1 1 1 3 3 1 1 1 1 1 3 1 1 1 1 1 3 1 1 1 3 1 1 1 3 1 1 1 3 1 1 1 3 2 3 1 1 1 3
##     [38] 1 1 3 3 3 1 1 1 3 1 1 1 1 3 3 1 1 3 1 1 1 3 3 1 1 1 1 1 1 1 1 1 1 3 1 3 1 1 1
##     [75] 3 1 1 1 2 1 1 1 3 1 3 2 3 3 1 3 1 1 1 3 1 3 1 1 2 1 3 2 3 1 1 1 1 1 1 1 3 3
##    [112] 3 1 1 1 1 1 3 1 1 1 1 1 3 1 1 1 1 1 1 1 3 3 3 1 1 1 2 3 3 3 3 1 1 1 1 1
##    [149] 1 3 1 2 1 1 1 3 1 1 3 3 3 3 1 1 1 1 1 3 1 3 1 1 1 1 1 3 3 3 1 1 3 3 1 3 1
```

```
## [186] 3 3 3 1 3 3 3 3 1 1 1 3 1 1 1 3 1 1 1 2 1 1 1 1 1 1 1 3 1 3 1 1 1 1 1
## [223] 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 1 3 3 1 3 3 1 1 1 1 3 1 3 3 3 1 1 1
## [260] 3 1 1 1 1 2 1 1 3 1 1 1 3 1 1 3 3 3 3 1 1 3 1 1 1 1 3 1 1 1 1 1 2 2
## [297] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [334] 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [371] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [408] 2 2 2 2 2 2 2 2 2 3 1 3 3 3 2 3 2 2 3 1 2 2 2 2 3 3 1 2 2 2 2 2 2 2
## [445] 3 2 3 2 3 3 2 2 2 3 2 3 2 2 1 2 2 3 2 3 2 2 1 2 2 3 3 2 2 2 3 2 2 1 2 3 2
## [482] 2 2 3 2 2 3 1 3 3 3 3 3 2 3 3 3 3 3 3 3 3 3 3 1 3 3 3 1 2 2 3 3 1 3 2 2 3
## [519] 3 2 3 3 3 3 3 1 1 3 3 2 3 3 3 2 3 3 2 2 3 3 2 3 3 3 3 3 1 1 3 3 3 3 3 3 3
## [556] 1 3 3 3 3 3 1 1 3 3 3 3 1 3 1 3 3 2 3 3 3 3 3 3 3 3 3 1 1 3 3 3 3 1 3 3 3
## [593] 1 3 3 3 3 3 1 3 3 3 3 3 3 3 3 3 3 3 1 1 2 3 3 3 3 3 1 3 3 3 1 3 3 3 1 3
## [630] 1 3 1 3 3 3 3 1 3 1 1 1 1 1 1 1 1 3 3 1 3 1 3 1 3 3 1 1 1 1 1 1 3 1 1 1 3
## [667] 3 3 1 3 1 1 1 3 2 3 3 1 1 1 3 3 3 1 1 1 1 3 1 3 3 1 1 1 1 1 3 1 3 1 3 1
## [704] 3 3 1 3 1 1 3 1 1 1 3 1 3 1 1 3 1 3 3 1 1 3 1 1 3 1 3 1 1 1 1 3 3 3 1 3 3
## [741] 3 1 1 3 1 1 1 3 3 1 3 3 1 3 3 1 1 3 1 1 3 1 3 1 1 3 1 1 1 1 3 1 1 3 1 1 3
## [778] 1 3 1 3 1 3 1 1 1 3 3 3 3 3 1 3 3 3 1 1 1 3 1 1 1 1 3 1 3 3 1 3 1 1 1 1 3
## [815] 1 3 3 3 1 3 1 3 1 1 1 3 1 3 1 1 3 1 1 1 1 1 1 3 3 3 3 1 1 3 3 3 1 3 1 1 1
## [852] 3 3 1 3 1 1 3 1 1 3 3 3 1 3 3 3 3 1 1 1 3 3 3 3 3 1 1 3 3 1 1 1 1 3 1 3 3
## [889] 3 3 1 1 1 3 1 3 3 1 3 1 1 1 3 1 1 1 1 3 3 3 3 3 1 1 3 1 1 3 1 1 1 1 3 3
## [926] 3 3 3 1 3 3 1 3 1 3 3 1 1 3 1 1 3 3 1 1 1 3 1 1 3 1 3 3 3 3 3 3 1 3 1 1
## [963] 3 3 3 1 3 1 1 1 3 1 1 1 1 1 1 1 3 3 3 1 1 1 3 3 1 1 1 1 3 1 1 1 3 1 3 3
## [1000] 1 3 3 1 1 1 3 3 1 3 1 3 3 1 3 3 1 1 1 1 1 1 1 1 3 3 1 1 1 3 1 3 1 1 3 1 1
## [1037] 1 1 1 3 3 3 3 1 1 3 3 1 3 1 3 1 1 1 1 1 1 3 3 3 3 3 3 1 3 1 1 1 3 1 3 3 1
## [1074] 1 3 1 3 1 3 1 3 3 3 3 1 1 3 1 3 1 3 1 1 3 3 1 3 1 1 1 3 1 1 1 1 3 1 1 1 1
## [1111] 3 3 1 1 3 1 3 1 3 1 3 1 3 3 1 1 1 1 1 1 3 2 3 1 1 1 3 3 3 3 1 1 1 3 3 3 1
## [1148] 1 1 3 1 3 3 1 3 1 1 3 3 3 3 1 3 3 1 1 3 3 1 3 1 1 3 3 1 1 1 3 1 1 3 3 1 3
## [1185] 3 1 3 2 1 1
##
## Within cluster sum of squares by cluster:
## [1] 1320.7098 747.5514 1708.2481
## (between_SS / total_SS = 36.5 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

K-means Clustering Visualisation

```
#create cluster plot and overlay target variable to compare clusters and presence or lack of heart disease
fviz_cluster(km.out, data=scaled.hd[1:5], geom = "point", shape = 20)+
  geom_point(aes(shape=scaled.hd$target, color=as.factor(km.out$cluster)))+
  labs(title = "Clusters and Heart Disease Status", shape = "Heart Disease Status")
```



```
table(km.out$cluster, scaled.hd[[6]])
```

```
##
##      0    1
## 1 396 136
## 2  29 157
## 3 136 336
```

Examining the visualisation of the clustering, the k-means clustering appears to be effective. The visualisation shows three clusters with little overlap. Clusters 1 and 3 and Clusters 1 and 2 have very limited overlap, while Clusters 2 and 3 have a little bit more, though not enough to be considered significant.

However, these clusters do not describe well the response variable (target, 0 = normal, 1 = heart disease). Examining the table of the cluster classification vs the target variable, approximately 20-30% of the observations belong to clusters other than that containing the majority of the observations with the target. In addition, the total variance of the dataset explained by the clustering can be determined by examining the results of the "Within cluster sum of squares by cluster". This value is 36.5%, which implies that the clustering does not provide a significant explanation of the response variable, and thus should not be used as a predictor of whether a patient has heart disease.